

RÉSUMÉ

Face à l'augmentation constante des cyberattaques et à l'interconnexion croissante des systèmes informatiques, le développement de systèmes de détection d'intrusions efficaces devient crucial pour la sécurité des infrastructures numériques. Cette recherche propose une approche innovante basée sur l'apprentissage automatique pour améliorer la détection et la classification des intrusions réseau.

Notre méthodologie adopte une stratégie hiérarchique en deux étapes appliquée au jeu de données NSL-KDD. La première phase réalise une classification binaire distinguant le trafic normal des activités malveillantes en combinant des algorithmes d'apprentissage supervisé avec un autoencodeur pour la détection d'anomalies. La seconde phase classe les intrusions détectées en quatre catégories : DoS, Probe, R2L et U2R. Pour traiter le déséquilibre des données, nous intégrons la technique SVM-SMOTE qui génère des échantillons synthétiques pour les classes minoritaires.

Les résultats expérimentaux démontrent la supériorité de l'autoencodeur pour la classification binaire avec une précision de 87,52%. L'application de SVM-SMOTE améliore significativement la détection des attaques sous-représentées, validant l'efficacité de notre approche hiérarchique. Cette recherche contribue au domaine de la cybersécurité en proposant une méthodologie robuste combinant apprentissage supervisé et non-supervisé pour répondre aux défis contemporains de la détection d'intrusions réseau.

Mots-clés : détection d'intrusion, apprentissage automatique, classification hiérarchique, NSL-KDD, autoencodeur, SVM-SMOTE, cybersécurité

ABSTRACT

With the exponential growth of cyber threats and the increasing interconnectivity of computer systems, developing effective intrusion detection mechanisms has become essential for protecting digital infrastructures. This research presents an innovative machine learning-based approach to enhance network intrusion detection and classification capabilities.

Our methodology employs a two-stage hierarchical strategy applied to the NSL-KDD dataset. The first stage performs binary classification to distinguish legitimate network traffic from malicious activities by combining supervised learning algorithms with an autoencoder designed for anomaly detection. The second stage categorizes detected intrusions into four attack types : DoS, Probe, R2L, and U2R. To address data imbalance issues, we integrate the SVM-SMOTE technique that generates synthetic samples for minority classes, ensuring better representation of underrepresented attack categories.

Experimental results demonstrate the superiority of the autoencoder approach for binary classification, achieving a remarkable accuracy of 87.52%. The implementation of SVM-SMOTE significantly improves detection rates for underrepresented attacks, validating the effectiveness of our hierarchical framework. This study contributes to the cybersecurity field by proposing a robust methodology that combines supervised and unsupervised learning techniques to address contemporary challenges in network intrusion detection.

Keywords : intrusion detection, machine learning, hierarchical classification, NSL-KDD, autoencoder, SVM-SMOTE, cybersecurity

REMERCIEMENTS

Au terme de ce travail de recherche, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire.

Mes premiers remerciements s'adressent à mes chers parents, pour leur amour inconditionnel, leur soutien constant et leurs sacrifices qui ont rendu possible la réussite de mon parcours universitaire. Leur confiance en moi et leurs encouragements ont été une source de motivation permanente tout au long de cette formation.

Je remercie chaleureusement mon directeur de mémoire, **Professeur EL MOUA-TASIM Abdelkrim**, pour son encadrement rigoureux, ses conseils précieux et sa disponibilité. Sa guidance scientifique, ses orientations méthodologiques et son expertise dans le domaine ont été déterminantes pour mener à bien cette recherche. Je lui suis reconnaissant pour la patience dont il a fait preuve et pour la confiance qu'il m'a accordée.

Mes sincères remerciements vont également à **Madame Salma GAOU**, responsable du département de Mathématiques et Informatique, pour son leadership, son soutien administratif et pour avoir créé un environnement d'apprentissage propice à l'épanouissement académique. Sa bienveillance et ses encouragements ont grandement facilité le déroulement de cette formation.

Je souhaite également exprimer ma gratitude à l'ensemble des professeurs du département de Mathématiques et Informatique qui ont contribué à ma formation durant ces deux années de master. Leur dévouement, la qualité de leurs enseignements et leur passion pour la transmission du savoir ont enrichi mes connaissances et développé mon esprit critique dans le domaine des mathématiques appliquées et de la science des données.

Enfin, je remercie tous mes collègues de promotion pour les moments d'échange et de collaboration qui ont marqué ce parcours, ainsi que toutes les personnes qui m'ont soutenu et encouragé durant cette période d'études.

À tous, mes sincères remerciements.

INTRODUCTION

À l'ère du numérique, les systèmes d'information constituent l'épine dorsale de nos sociétés modernes. Des entreprises multinationales aux institutions gouvernementales, en passant par les services publics essentiels, l'ensemble de notre infrastructure économique et sociale repose désormais sur des réseaux informatiques complexes et interconnectés. Cette dépendance croissante aux technologies de l'information, bien qu'elle offre des opportunités sans précédent en termes d'efficacité et de connectivité, expose simultanément nos systèmes à des risques de sécurité considérables.

Les statistiques récentes révèlent une augmentation alarmante des cyberattaques à l'échelle mondiale. Selon les rapports de cybersécurité, le nombre d'incidents de sécurité informatique a triplé au cours de la dernière décennie, avec des conséquences financières se chiffrant en milliards d'euros annuellement. Les attaques deviennent progressivement plus sophistiquées, exploitant des vulnérabilités techniques avancées et adoptant des stratégies d'intrusion de plus en plus subtiles. Face à cette menace persistante et évolutive, la communauté scientifique et industrielle s'efforce de développer des mécanismes de défense innovants et efficaces.

Dans ce contexte, les systèmes de détection d'intrusions (IDS - Intrusion Detection Systems) émergent comme une composante fondamentale de l'architecture de sécurité moderne. Ces systèmes ont pour mission de surveiller en continu le trafic réseau, d'identifier les comportements anormaux et de signaler les tentatives d'intrusion potentielles. Traditionnellement, les IDS s'appuyaient sur des approches basées sur des signatures, comparant les patterns de trafic observés à des bases de données d'attaques connues. Cependant, cette méthodologie présente des limitations importantes, notamment son incapacité à détecter les nouvelles formes d'attaques (zero-day attacks) et sa dépendance à des mises à jour fréquentes des bases de signatures.

L'émergence de l'apprentissage automatique et de l'intelligence artificielle ouvre de nouvelles perspectives prometteuses pour surmonter ces défis. Les techniques d'apprentissage automatique offrent la capacité d'apprendre automatiquement à partir des données historiques, de généraliser à partir d'exemples et de s'adapter aux nouveaux types de menaces sans intervention humaine explicite. Cette approche data-driven présente l'avantage de pouvoir identifier des patterns complexes et subtils dans le trafic réseau, potentiellement imperceptibles par les méthodes traditionnelles.

Les approches d'apprentissage automatique appliquées à la détection d'intrusions se déclinent selon plusieurs paradigmes. L'apprentissage supervisé exploite des jeux de données étiquetés pour entraîner des modèles capables de classer le trafic réseau en catégories prédéfinies. Les algorithmes comme les forêts aléatoires, les machines à vecteurs de support (SVM) et les réseaux de neurones ont démontré leur efficacité dans ce domaine. Parallèlement, l'apprentissage non-supervisé, notamment à travers les techniques de détection d'anomalies, permet d'identifier des comportements atypiques sans connaissance préalable des types d'attaques, offrant ainsi une capacité de détection proactive.

Cependant, l'application de l'apprentissage automatique à la cybersécurité soulève des défis techniques spécifiques. Le premier défi concerne la qualité et la représentativité des données d'entraînement. Les jeux de données de cybersécurité souffrent fréquemment d'un déséquilibre prononcé entre les classes, où les échantillons d'attaques représentent une fraction minime du trafic total. Cette asymétrie peut conduire à des modèles biaisés privilégiant la détection du trafic normal au détriment de la sensibilité aux intrusions. Le second défi réside dans la nature évolutive des menaces cybernétiques. Les attaquants adaptent continuellement leurs stratégies, développant de nouvelles techniques pour contourner les systèmes de détection existants, ce qui nécessite des modèles capables d'évoluer et de s'adapter dynamiquement.

La littérature scientifique révèle diverses approches méthodologiques pour adresser ces problématiques. Certaines recherches se concentrent sur l'amélioration des algorithmes de classification individuels, optimisant leurs hyperparamètres ou développant de nouvelles architectures. D'autres explorent les techniques d'ensemble, combinant plusieurs modèles pour améliorer la robustesse et la précision de la détection. Une troisième voie d'investigation porte sur le préprocessing des données, incluant la sélection de caractéristiques, la réduction de dimensionnalité et les techniques de rééquilibrage des classes.

Parmi les jeux de données de référence utilisés pour évaluer les systèmes de détection d'intrusions, NSL-KDD occupe une position particulière. Héritier du célèbre dataset KDD Cup 99, NSL-KDD corrige plusieurs limitations de son prédécesseur, notamment en éliminant les enregistrements redondants et en proposant une distribution plus équilibrée des classes. Ce dataset comprend une variété d'attaques organisées en quatre catégories principales : les attaques par déni de service (DoS), les tentatives de reconnaissance ou sondage (Probe), les intrusions de type distant vers local (R2L), et les élévations de privilèges (U2R). Cette taxonomie reflète la diversité des menaces rencontrées dans les environnements réseau réels.

Face aux défis identifiés et en s'appuyant sur l'état de l'art existant, ce mémoire propose une approche innovante basée sur une stratégie hiérarchique d'apprentissage automatique pour la détection d'intrusions réseau. Notre méthodologie se distingue par l'adoption d'une architecture en deux étapes complémentaires, permettant une analyse progressive et affinée du trafic réseau. Cette approche hiérarchique vise à optimiser à la fois la précision de détection et la capacité de classification fine des différents types d'intrusions.

La première étape de notre système implémente une classification binaire robuste distinguant le trafic légitime des activités suspectes. Cette phase mobilise un ensemble diversifié d'algorithmes d'apprentissage supervisé, incluant les arbres de décision, les forêts aléatoires, les machines à vecteurs de support et les réseaux de neurones multicouches. En complément de ces approches classiques, nous intégrons un autoencodeur spécialement conçu pour la détection d'anomalies, exploitant les capacités de l'apprentissage non-supervisé pour identifier les patterns atypiques dans le trafic réseau.

La seconde étape affine la classification en catégorisant les intrusions détectées selon les quatre typologies reconnues du dataset NSL-KDD. Pour surmonter le défi du déséquilibre des classes, particulièrement critique pour les attaques de type R2L et U2R, nous implémentons la technique SVM-SMOTE (Support Vector Machine - Synthetic Minority Oversampling Technique). Cette méthode génère des échantillons synthétiques pour les classes minoritaires, améliorant ainsi la représentativité des données d'entraînement et la capacité de généralisation des modèles.

Les contributions principales de cette recherche s'articulent autour de trois axes majeurs. Premièrement, nous démontrons l'efficacité d'une approche hiérarchique combinant apprentissage supervisé et non-supervisé pour la détection d'intrusions. Deuxièmement, nous évaluons de manière comparative les performances de différents algorithmes d'apprentissage automatique dans ce contexte spécifique. Troisièmement, nous validons l'impact positif des techniques de rééquilibrage des données sur la détection des classes minoritaires critiques.

Ce mémoire s'organise en quatre sections principales. Après cette introduction contextuelle, la section méthodologie détaille notre approche hiérarchique, les algorithmes implémentés et les techniques de préprocessing appliquées. La section résultats présente une évaluation exhaustive des performances obtenues, incluant des analyses comparatives et des métriques de validation. Enfin, la section discussion interprète ces résultats, examine leurs implications pour la cybersécurité et propose des perspectives d'amélioration future.

L'objectif ultime de cette recherche est de contribuer à l'avancement des systèmes de détection d'intrusions en proposant une méthodologie robuste, reproductible et adaptable aux environnements réseau contemporains. À travers une validation rigoureuse sur le dataset NSL-KDD, nous aspirons à démontrer la viabilité et l'efficacité de notre approche pour répondre aux défis actuels de la cybersécurité.

1 MÉTHODOLOGIE

Cette section présente en détail l'approche méthodologique développée pour la détection d'intrusions réseau. Notre stratégie repose sur une architecture hiérarchique combinant apprentissage supervisé et non-supervisé, appliquée au jeu de données NSL-KDD. L'ensemble du processus, depuis le prétraitement des données jusqu'à la classification finale, est conçu pour optimiser à la fois la précision de détection et la capacité de discrimination entre les différents types d'attaques.

1.1 Architecture générale du système

Notre système de détection d'intrusions adopte une approche hiérarchique en deux étapes distinctes, illustrée dans la Figure 1. Cette architecture permet une analyse progressive du trafic réseau, commençant par une classification binaire robuste avant d'affiner la caractérisation des menaces détectées.



FIGURE 1 – Architecture générale du système de détection d'intrusions hiérarchique

La première étape effectue une discrimination fondamentale entre le trafic normal et les activités suspectes. Cette phase mobilise plusieurs algorithmes d'apprentissage

automatique, incluant des approches supervisées traditionnelles et une méthode non-supervisée basée sur un autoencodeur. La seconde étape se concentre sur la classification fine des intrusions détectées, catégorisant les attaques selon quatre typologies reconnues dans la littérature scientifique.

1.2 Jeu de données NSL-KDD

1.2.1 Présentation et structure

Le jeu de données NSL-KDD constitue une version améliorée du célèbre dataset KDD Cup 99, corrigeant plusieurs limitations de son prédécesseur. Ce dataset comprend 125,973 enregistrements d'entraînement et 22,544 enregistrements de test, chaque instance étant caractérisée par 41 attributs décrivant les propriétés d'une connexion réseau.

1.2.2 Description détaillée des 41 caractéristiques

Les caractéristiques du dataset NSL-KDD se répartissent en trois catégories principales, chacune capturant des aspects différents du comportement réseau :

Caractéristiques de base (9 attributs) Ces attributs sont directement extraits des en-têtes de paquets TCP/IP et représentent les propriétés fondamentales d'une connexion :

- **duration** : Durée de la connexion en secondes
- **protocol_type** : Type de protocole utilisé (TCP, UDP, ICMP)
- **service** : Service réseau de destination (HTTP, FTP, SMTP, etc.)
- **flag** : État normal ou d'erreur de la connexion (SF, S0, REJ, etc.)
- **src_bytes** : Nombre d'octets de données transférés de la source vers la destination
- **dst_bytes** : Nombre d'octets de données transférés de la destination vers la source
- **land** : 1 si la connexion provient et va vers le même hôte/port, 0 sinon
- **wrong_fragment** : Nombre de fragments incorrects
- **urgent** : Nombre de paquets urgents

Caractéristiques de trafic (13 attributs) Ces statistiques sont calculées sur une fenêtre glissante de 100 connexions et analysent les patterns de comportement :

Statistiques "même hôte" :

- **count** : Nombre de connexions vers le même hôte que la connexion courante
- **srv_count** : Nombre de connexions vers le même service que la connexion courante
- **serror_rate** : Pourcentage de connexions ayant une erreur SYN
- **srv_serror_rate** : Pourcentage de connexions vers le même service ayant une erreur SYN
- **rerror_rate** : Pourcentage de connexions ayant une erreur REJ
- **srv_rerror_rate** : Pourcentage de connexions vers le même service ayant une erreur REJ
- **same_srv_rate** : Pourcentage de connexions vers le même service
- **diff_srv_rate** : Pourcentage de connexions vers différents services

Statistiques "hôte de destination" :

- **dst_host_count** : Nombre de connexions ayant le même hôte de destination
- **dst_host_srv_count** : Nombre de connexions ayant le même service de destination
- **dst_host_same_srv_rate** : Pourcentage de connexions vers le même service de destination
- **dst_host_diff_srv_rate** : Pourcentage de connexions vers différents services de destination
- **dst_host_same_src_port_rate** : Pourcentage de connexions utilisant le même port source

Caractéristiques de contenu (19 attributs) Ces attributs analysent le contenu des paquets et sont particulièrement utiles pour détecter les attaques R2L et U2R :

Indicateurs d'accès privilégié :

- **hot** : Nombre d'indicateurs "hot" (accès à fichiers/répertoires sensibles)
- **num_failed_logins** : Nombre de tentatives de connexion échouées
- **logged_in** : 1 si connecté avec succès, 0 sinon
- **num_compromised** : Nombre de conditions "compromised"
- **root_shell** : 1 si un shell root a été obtenu, 0 sinon
- **su_attempted** : 1 si une commande "su root" a été tentée, 0 sinon

Statistiques d'accès aux fichiers :

- **num_root** : Nombre d'accès "root"
- **num_file_creations** : Nombre d'opérations de création de fichiers
- **num_shells** : Nombre d'invitations de commandes shell
- **num_access_files** : Nombre d'opérations d'accès aux fichiers de contrôle
- **num_outbound_cmds** : Nombre de commandes sortantes dans une session FTP

Indicateurs de session :

- **is_host_login** : 1 si le login appartient à la liste "hot", 0 sinon
- **is_guest_login** : 1 si le login est "guest", 0 sinon

Statistiques d'erreur de destination :

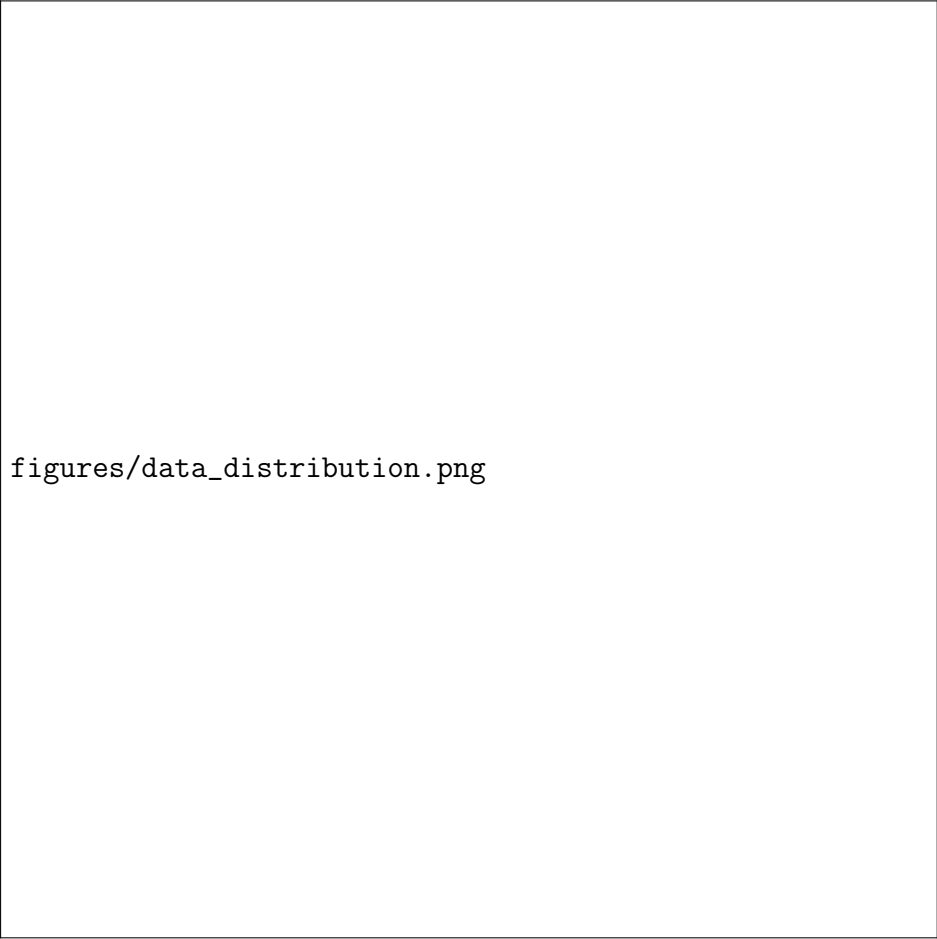
- **dst_host_srv_diff_host_rate** : Pourcentage de connexions vers différents hôtes
- **dst_host_serror_rate** : Pourcentage de connexions ayant une erreur SYN
- **dst_host_srv_serror_rate** : Pourcentage de connexions vers le même service ayant une erreur SYN
- **dst_host_rerror_rate** : Pourcentage de connexions ayant une erreur REJ
- **dst_host_srv_rerror_rate** : Pourcentage de connexions vers le même service ayant une erreur REJ

1.2.3 Taxonomie des attaques

NSL-KDD organise les intrusions selon quatre catégories principales, chacune représentant un mode opératoire distinct :

TABLE 1 – Distribution des catégories d’attaques dans NSL-KDD

Catégorie	Description	Exemples
DoS	Attaques par déni de service visant à saturer les ressources système	neptune, smurf, pod
Probe	Tentatives de reconnaissance pour découvrir les vulnérabilités	satan, portsweep, nmap
R2L	Intrusions distantes cherchant un accès local non autorisé	warezclient, imap, spy
U2R	Élévations de privilèges d’utilisateur normal vers administrateur	buffer_overflow, rootkit



figures/data_distribution.png

FIGURE 2 – Distribution des classes dans le jeu de données NSL-KDD

1.3 Prétraitement des données

1.3.1 Encodage des variables catégorielles

Le jeu de données NSL-KDD contient trois attributs catégoriels : le type de protocole, le service réseau et l’état de la connexion. Ces variables textuelles nécessitent une transformation numérique pour être exploitables par les algorithmes d’apprentissage automatique.

Nous appliquons un encodage par fréquence (Label Frequency Encoding) qui attribue des valeurs numériques en fonction de la fréquence d'apparition de chaque catégorie. Pour une variable catégorielle X avec des catégories $\{c_1, c_2, \dots, c_k\}$, l'encodage par fréquence assigne à chaque catégorie c_i la valeur :

$$\text{encoded}(c_i) = \text{rank}(\text{freq}(c_i)) \quad (1)$$

où $\text{freq}(c_i)$ représente la fréquence d'apparition de la catégorie c_i et rank attribue un rang basé sur cette fréquence.

1.3.2 Normalisation des données

La standardisation des caractéristiques numériques constitue une étape cruciale pour assurer l'efficacité des algorithmes d'apprentissage automatique. Nous appliquons la standardisation Z-score :

$$z_i = \frac{x_i - \mu}{\sigma} \quad (2)$$

où x_i représente la valeur originale, μ la moyenne arithmétique et σ l'écart-type de la caractéristique considérée.

1.4 Algorithmes d'apprentissage automatique

1.4.1 Arbres de décision

Les arbres de décision constituent une famille d'algorithmes d'apprentissage supervisé particulièrement adaptée à l'interprétabilité. Un arbre de décision partitionne récursivement l'espace des caractéristiques en utilisant des tests binaires sur les attributs.

Critère de division Pour construire l'arbre, nous utilisons le critère d'entropie. L'entropie d'un ensemble S est définie par :

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (3)$$

où c est le nombre de classes et p_i la proportion d'exemples de classe i dans S .

Le gain d'information pour un attribut A divisant S en sous-ensembles $\{S_1, S_2, \dots, S_v\}$ est :

$$\text{Gain}(S, A) = H(S) - \sum_{j=1}^v \frac{|S_j|}{|S|} H(S_j) \quad (4)$$

Critère d'arrêt L'algorithme s'arrête lorsque :

- Tous les exemples appartiennent à la même classe
- La profondeur maximale est atteinte
- Le nombre minimum d'exemples par feuille est atteint
- Le gain d'information est inférieur à un seuil prédéfini

1.4.2 Forêts aléatoires

Les forêts aléatoires combinent plusieurs arbres de décision selon le principe du bagging (Bootstrap AGGREGatING). Cette méthode d'ensemble améliore la robustesse et la généralisation.

Algorithme de construction Pour construire une forêt de T arbres :

1. Pour $t = 1$ à T :
 - Générer un échantillon bootstrap S_t de taille n avec remise
 - À chaque nœud, sélectionner aléatoirement m caractéristiques parmi les p disponibles (généralement $m = \sqrt{p}$)
 - Construire l'arbre h_t en utilisant S_t et les m caractéristiques sélectionnées

Prédiction La prédiction finale est obtenue par vote majoritaire pour la classification :

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x})\} \quad (5)$$

La probabilité de classe k est estimée par :

$$\hat{p}_k(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = k) \quad (6)$$

où $\mathbb{I}$ est la fonction indicatrice.

1.4.3 Machines à vecteurs de support (SVM)

Les SVM recherchent l'hyperplan optimal séparant les classes en maximisant la marge entre les points les plus proches de chaque classe.

Formulation mathématique Pour un problème de classification binaire avec des données linéairement séparables, l'objectif est de trouver l'hyperplan $\mathbf{w}^T \mathbf{x} + b = 0$ qui maximise la marge.

Le problème d'optimisation s'écrit :

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (7)$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \quad (8)$$

Cas non-linéairement séparable Pour traiter les cas non-linéairement séparables, nous introduisons des variables de relaxation ξ_i :

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad (9)$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad (10)$$

$$\xi_i \geq 0, \quad i = 1, \dots, n \quad (11)$$

où C est le paramètre de régularisation contrôlant le compromis entre la maximisation de la marge et la minimisation des erreurs de classification.

Noyau RBF Pour capturer les relations non-linéaires, nous utilisons le noyau gaussien (RBF) :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (12)$$

où γ contrôle l'influence de chaque exemple d'entraînement.

1.4.4 Réseaux de neurones multi-couches (MLP)

Le perceptron multi-couches est un réseau de neurones feedforward capable d'apprendre des relations non-linéaires complexes.

Architecture Notre MLP comprend :

- Couche d'entrée : 41 neurones
- Couche cachée : 100 neurones avec activation ReLU
- Couche de sortie : 2 neurones avec activation softmax (classification binaire)

Fonction d'activation ReLU La fonction ReLU (Rectified Linear Unit) est définie par :

$$\text{ReLU}(x) = \max(0, x) \quad (13)$$

Fonction softmax Pour la couche de sortie, la fonction softmax normalise les sorties en probabilités :

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (14)$$

Propagation avant Pour une couche l , le calcul s'effectue selon :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (15)$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) \quad (16)$$

où $\mathbf{W}^{(l)}$ et $\mathbf{b}^{(l)}$ sont respectivement la matrice de poids et le vecteur de biais de la couche l , et f est la fonction d'activation.

Fonction de coût Nous utilisons l'entropie croisée comme fonction de coût :

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (17)$$

Rétropropagation Les gradients sont calculés par rétropropagation :

$$\frac{\partial J}{\partial \mathbf{W}^{(l)}} = \frac{1}{m} \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T \quad (18)$$

$$\frac{\partial J}{\partial \mathbf{b}^{(l)}} = \frac{1}{m} \sum_{i=1}^m \boldsymbol{\delta}^{(l)} \quad (19)$$

où $\boldsymbol{\delta}^{(l)}$ est l'erreur de la couche l .

1.4.5 AdaBoost

AdaBoost (Adaptive Boosting) est une méthode d'ensemble qui combine séquentiellement des classificateurs faibles en se concentrant sur les exemples difficiles à classifier.

Algorithme AdaBoost

1. Initialiser les poids : $w_1^{(i)} = \frac{1}{m}$ pour $i = 1, \dots, m$
2. Pour $t = 1$ à T :
 - (a) Entraîner un classificateur faible h_t sur la distribution w_t
 - (b) Calculer l'erreur pondérée :

$$\epsilon_t = \sum_{i=1}^m w_t^{(i)} \mathbb{I}(y_i \neq h_t(\mathbf{x}_i)) \quad (20)$$

- (c) Calculer le coefficient :

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (21)$$

- (d) Mettre à jour les poids :

$$w_{t+1}^{(i)} = \frac{w_t^{(i)} \exp(-\alpha_t y_i h_t(\mathbf{x}_i))}{Z_t} \quad (22)$$

où Z_t est le facteur de normalisation

3. Le classificateur final est :

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right) \quad (23)$$

1.5 Architecture des modèles spécialisés

1.5.1 Autoencodeur pour la détection d'anomalies

L'autoencodeur constitue l'élément central de notre approche non-supervisée pour la classification binaire.

figures/autoencoder_architecture.png

FIGURE 3 – Architecture de l’autoencodeur pour la détection d’anomalies

Architecture Notre autoencodeur se compose de :

- **Couche d’entrée** : 41 neurones avec bruit gaussien $\mathcal{N}(0, 0.15^2)$
- **Couche d’encodage** : 15 neurones avec activation SELU et dropout (5%)
- **Couche de décodage** : 41 neurones avec activation SELU

Fonction d’activation SELU La fonction SELU (Scaled Exponential Linear Unit) est définie par :

$$\text{SELU}(x) = \lambda \begin{cases} x & \text{si } x > 0 \\ \alpha(e^x - 1) & \text{si } x \leq 0 \end{cases} \quad (24)$$

avec $\alpha = 1.6732632423543772$ et $\lambda = 1.0507009873554805$.

Fonction de coût L’erreur quadratique moyenne (MSE) est utilisée comme fonction de coût :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (25)$$

Détection d’anomalies L’erreur de reconstruction pour un échantillon $\mathbf{x}$ est :

$$\text{erreur_reconstruction}(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \quad (26)$$

Le seuil de détection est défini comme le 95ème percentile des erreurs de reconstruction sur les données normales :

$$\tau = \text{percentile}_{95}(\{\text{erreur_reconstruction}(\mathbf{x}_i) : y_i = \text{normal}\}) \quad (27)$$

1.5.2 Réseau de neurones profond pour classification multi-classes

Architecture Le DNN comprend :

- **Couche d'entrée** : 41 neurones
- **Première couche cachée** : 80 neurones, ReLU, dropout (30%)
- **Seconde couche cachée** : 40 neurones, ReLU, dropout (20%)
- **Couche de sortie** : 4 neurones, softmax

Fonction de coût L'entropie croisée sparse est utilisée :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log(p_{y_i}) \quad (28)$$

1.6 Stratégie hiérarchique

1.6.1 Étape 1 : Classification binaire

La première étape distingue le trafic normal des activités malveillantes. Nous évaluons tous les algorithmes présentés et sélectionnons le plus performant.

1.6.2 Étape 2 : Classification des types d'attaques

Cette étape utilise uniquement les échantillons classifiés comme malveillants, permettant une spécialisation sur la discrimination entre types d'attaques.

1.7 Gestion du déséquilibre : SVM-SMOTE

1.7.1 Algorithme SVM-SMOTE

SVM-SMOTE améliore SMOTE en utilisant un SVM pour identifier les échantillons critiques :

1. Entraîner un SVM sur les données déséquilibrées
2. Identifier les vecteurs de support de la classe minoritaire
3. Pour chaque vecteur de support $\mathbf{x}_i$:
 - (a) Trouver ses k plus proches voisins de la même classe
 - (b) Générer des échantillons synthétiques par interpolation :

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_j - \mathbf{x}_i) \quad (29)$$

où $\lambda \sim \mathcal{U}(0, 1)$ et $\mathbf{x}_j$ est un voisin choisi aléatoirement



figures/smote_illustration.png

FIGURE 4 – Illustration du processus SVM-SMOTE

1.8 Métriques d'évaluation

1.8.1 Métriques de base

$$\text{Précision} = \frac{TP}{TP + FP} \quad (30)$$

$$\text{Rappel} = \frac{TP}{TP + FN} \quad (31)$$

$$\text{F1-score} = \frac{2 \times \text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (32)$$

$$\text{Exactitude} = \frac{TP + TN}{TP + TN + FP + FN} \quad (33)$$

1.8.2 Métriques agrégées

$$\text{F1-macro} = \frac{1}{K} \sum_{k=1}^K \text{F1-score}_k \quad (34)$$

$$\text{F1-micro} = \frac{2 \times \text{Précision}_{\text{micro}} \times \text{Rappel}_{\text{micro}}}{\text{Précision}_{\text{micro}} + \text{Rappel}_{\text{micro}}} \quad (35)$$

Cette méthodologie complète assure une évaluation rigoureuse et reproductible de notre système de détection d'intrusions hiérarchique.

2 RESULTATS

3 DISCUSSION